# Software Engineering

Complete Exam Notes — Introduction · SDLC · Agile · COCOMO · Requirements

| | |
|---|---|
| **Subject** | Introduction to Software Engineering |
| **Topics Covered** | 5 Units — All Major Questions |
| **Purpose** | University Exam Preparation |
| **Style** | Point-by-Point Answers |

# UNIT 1 — Introduction to Software Engineering

## Q1. Define the term Software Engineering.

Software Engineering is the systematic, disciplined, and quantifiable approach to the development, operation, and maintenance of software. It applies engineering principles to software creation to produce reliable, efficient, and maintainable software within time and budget constraints.

**Key Definitions:**

- IEEE Definition: 'The application of a systematic, disciplined, quantifiable approach to the development, operation and maintenance of software.'
- Fritz Bauer: 'The establishment and use of sound engineering principles to obtain economically software that is reliable and works efficiently on real machines.'

**Goals of Software Engineering:**

- Produce high-quality, reliable software.
- Deliver within time and cost constraints.
- Meet user requirements and be maintainable.

---

## Q2. What do you understand by Exploratory Style Programming?

Exploratory Style Programming (also called 'Cut and Fix' or ad-hoc programming) is an informal, unstructured approach to software development where a programmer directly starts coding without any prior planning, design, or documentation.

**Characteristics:**

- No formal requirements or design phase.
- Code is written and then fixed repeatedly until it appears to work.
- No project planning or management involved.
- Suitable only for very small, personal programs.
- Developer and user are usually the same person.

---

## Q3. List the drawbacks of Exploratory Style Programming.

- Lack of Structure: No systematic approach; code becomes messy and hard to maintain.
- Poor Scalability: Cannot handle large or complex projects.
- No Documentation: Future maintenance becomes extremely difficult.
- Unreliable Software: Frequent bugs due to absence of testing strategies.
- Cost Overruns: Repeated fixing without planning leads to wasted effort and cost.
- No Team Coordination: Cannot support collaborative development.
- Difficult to Reuse: Code is written for a specific case and cannot be reused easily.
- No Quality Control: No standards are followed, resulting in poor quality software.

---

## Q4. Differentiate between Exploratory Programming and Software Engineering.

| Exploratory Programming | Software Engineering |
|---|---|
| No planning or design phase | Follows systematic planning and design |
| No documentation | Thorough documentation at every stage |
| Individual effort only | Supports team-based development |

| Exploratory Programming | Software Engineering |
|---|---|
| Code is written and fixed repeatedly | Follows SDLC phases with quality control |
| Not scalable for large systems | Designed for large, complex systems |
| No cost/time estimation | Proper cost and time estimation done |
| No testing strategy | Formal testing and validation phases included |
| Suitable for tiny personal programs | Suitable for commercial/enterprise software |

## Q5. What is meant by Software Complexity? Explain with suitable examples.

Software Complexity refers to the degree of difficulty involved in understanding, developing, modifying, or maintaining a software system. It arises due to the intricate interactions among components, large codebase, or unclear requirements.

**Types of Software Complexity:**

- Problem Complexity: The inherent difficulty of the problem itself.
- Algorithmic Complexity: The time/space requirements of algorithms used.
- Structural Complexity: Complexity arising from the architecture and design.
- Cognitive Complexity: How hard it is for a developer to understand the code.

**Examples:**

- Air Traffic Control System: Thousands of aircraft, real-time data, safety-critical decisions — extremely complex interactions.
- Banking Software: Handles millions of transactions, security, regulations, concurrent users — structural and algorithmic complexity.
- Operating System: Manages hardware, processes, memory simultaneously — highest level of complexity.

■ *Complexity is measured using metrics like Cyclomatic Complexity (McCabe's metric), Lines of Code (LOC), and Halstead's Complexity Measures.*

## Q6. Describe the basic principles of Software Engineering.

- Separation of Concerns: Divide a large problem into smaller manageable parts.
- Modularity: Design software in independent, interchangeable modules.
- Abstraction: Hide implementation details; expose only what is necessary.
- Anticipation of Change: Design software to accommodate future changes easily.
- Generality: Build general-purpose solutions where possible.
- Incrementality: Develop software in small, manageable increments.
- Rigor and Formality: Follow defined processes, standards, and documentation.
- Software Reuse: Reuse existing components to reduce cost and effort.

## Q7. What are the various types of software projects?

- Organic Projects: Small teams, familiar problem domain, simple requirements. e.g., Payroll systems.
- Semi-Detached Projects: Mixed experience teams, moderately complex. e.g., Database management systems.
- Embedded Projects: Hardware-integrated, real-time, complex requirements. e.g., ATM software, air traffic control.
- Business Applications: ERP, CRM systems used in enterprise environments.
- System Software: OS, compilers, drivers — interact directly with hardware.
- Scientific/Engineering Software: Used for numerical computation and simulation.

• Web Applications: Browser-based, networked, dynamic systems.

## Q8. Differentiate between Software Product and Software Service.

| Software Product | Software Service |
|---|---|
| A tangible software artifact sold to customers | Software capabilities delivered over a network |
| Installed locally by the user | Accessed remotely; hosted by provider |
| One-time purchase or license fee | Subscription or pay-per-use model |
| Updates require manual installation | Updates deployed automatically by provider |
| Example: Microsoft Office (boxed) | Example: Google Docs, Salesforce CRM |
| User owns a copy | User accesses but does not own a copy |

## Q9. What is Software Crisis?

The Software Crisis refers to a period (late 1960s–1980s) when the software industry faced severe difficulties in delivering software on time, within budget, and to the required quality standards. The term was coined at the 1968 NATO Software Engineering Conference.

**Symptoms of Software Crisis:**

- Projects were delivered late or never completed.
- Software costs far exceeded initial budgets.
- Software was unreliable and difficult to maintain.
- Software did not meet user requirements.
- Poor software quality caused system failures.

## Q10. Explain the reasons behind Software Crisis.

- Rapidly Increasing Complexity: Hardware became more powerful, demanding more complex software than developers could handle.
- Lack of Formal Methods: No systematic approach to software development existed.
- Poor Estimation: Inability to accurately estimate time and cost led to overruns.
- Changing Requirements: Frequent requirement changes mid-development caused failures.
- Lack of Documentation: Absence of proper documentation made maintenance impossible.
- Untrained Workforce: Developers lacked formal training in software engineering principles.
- No Reusability: Every project started from scratch, wasting effort.

## Q11. Discuss how software design techniques have evolved.

- 1950s–60s: Machine-level and assembly language programming. No formal design.
- 1960s–70s: Structured Programming introduced (Dijkstra) — use of control structures (loops, if-else); top-down design.
- 1970s–80s: Data Flow Design and Structured Design (Yourdon-Constantine). Introduction of DFDs and structure charts.
- 1980s–90s: Object-Oriented Design (OOD) emerged — encapsulation, inheritance, polymorphism (Booch, Rumbaugh, Jacobson → UML).
- 1990s–2000s: Component-Based Design and Design Patterns (Gang of Four). Reuse became central.
- 2000s–present: Service-Oriented Architecture (SOA), Microservices, Agile design practices, Domain-Driven Design (DDD).

# UNIT 2 — SDLC and Process Models

## Q12. What is SDLC?

SDLC (Software Development Life Cycle) is a structured process used to design, develop, test, and deliver software. It provides a well-defined sequence of phases that guide a project from inception to deployment and maintenance.

---

## Q13. List the different phases of SDLC.

- 1. Requirement Analysis — Gather and document user requirements.
- 2. Feasibility Study — Assess technical, financial, and operational feasibility.
- 3. System Design — Create architecture, data models, interface design.
- 4. Implementation (Coding) — Write the actual source code.
- 5. Testing — Verify and validate the software.
- 6. Deployment — Release software to the production environment.
- 7. Maintenance — Fix bugs, add features, improve performance.

---

## Q14. Define SDLC and explain its purpose.

SDLC is a framework that defines the tasks performed at each step of software development.

**Purpose of SDLC:**

- Provides a structured methodology to manage complex projects.
- Ensures software is developed systematically with clear milestones.
- Helps control quality at every stage via reviews and testing.
- Enables accurate planning of cost, time, and resources.
- Facilitates communication among developers, testers, and stakeholders.

---

## Q15. Explain Verification and Validation in software.

| Verification | Validation |
|---|---|
| Are we building the product right? | Are we building the right product? |
| Checks that the software conforms to specifications | Checks that the software meets user needs |
| Done without executing the code (static testing) | Done by executing the code (dynamic testing) |
| Involves reviews, walkthroughs, inspections | Involves black-box and system testing |
| Done during development phase | Done after development is complete |
| Example: Code review, design inspection | Example: User acceptance testing (UAT) |

---

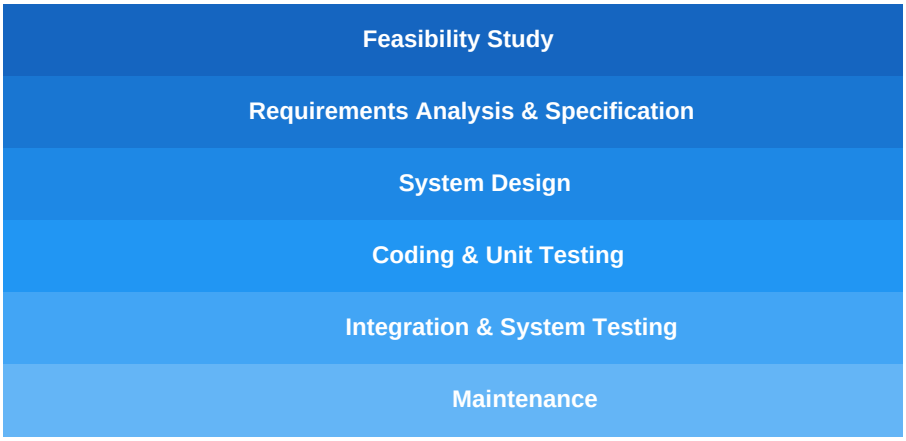
## Q16. Explain the Classical Waterfall Model with a neat diagram.

The Classical Waterfall Model is the simplest and oldest SDLC model. Phases flow sequentially downward like a waterfall — each phase must be completed before the next begins.

**Phases (in order):**

- 1. Feasibility Study
- 2. Requirements Analysis and Specification
- 3. System Design

- 4. Coding and Unit Testing
- 5. Integration and System Testing
- 6. Maintenance




---

## Q17. State the advantages and disadvantages of the Waterfall Model.

**Advantages:**

- Simple and easy to understand and use.
- Works well for small projects with clearly defined requirements.
- Phases are clearly defined with specific deliverables and milestones.
- Easy to manage due to rigidity of the model.

**Disadvantages:**

- No working software produced until late in the lifecycle.
- Cannot accommodate changing requirements.
- High risk and uncertainty for complex projects.
- Poor model for long, ongoing projects.
- Difficult to go back to a previous phase once completed.

---

## Q18. Explain the Iterative Waterfall Model.

The Iterative Waterfall Model is an improvement over the classical waterfall model. It allows feedback loops between adjacent phases, enabling correction of errors discovered in a later phase by returning to the previous phase.

**Key Features:**

- Feedback loops between consecutive phases (not non-adjacent).
- Errors detected in one phase can be corrected by revisiting the previous phase.
- Reduces risk of discovering errors very late in the project.
- Still sequential in nature but more flexible than classical waterfall.

■ *Example: If an error is found during coding, developers can go back to design and fix it before continuing.*

---

## Q19. Explain the V-Model in SDLC.

The V-Model (Verification and Validation Model) is an extension of the Waterfall model. Each development phase on the left side of the 'V' has a corresponding testing phase on the right side.

**Left side (Development):**

- Requirements → Acceptance Testing
- System Design → System Testing
- Architectural Design → Integration Testing

- Module Design → Unit Testing
- Coding (bottom of V)

**Advantages of V-Model:**

- Testing is planned early alongside development.
- Higher chance of success due to early test planning.
- Simple and easy to manage.

**Disadvantages:**

- Rigid — no flexibility for requirement changes.
- No early working prototype.

---

## Q20. Compare Waterfall Model and V-Model.

| Waterfall Model | V-Model |
|---|---|
| Testing done after all development phases | Testing planned parallel to development phases |
| No explicit mapping of tests to phases | Each phase has a corresponding test phase |
| Less focus on verification during development | Strong focus on both verification and validation |
| Lower cost to use | Higher cost due to parallel test planning |
| Suitable for simple, stable requirements | Suitable for safety-critical systems |

---

## Q21. Explain the Prototyping Model with diagram.

In the Prototyping Model, a working prototype (partial version) of the software is built early to help clarify requirements and get user feedback before full development begins.

**Steps:**

- 1. Gather initial requirements.
- 2. Design and build a quick prototype.
- 3. User evaluates the prototype.
- 4. Refine the prototype based on feedback.
- 5. Repeat until the prototype is approved.
- 6. Develop the final system.

**Gather Requirements → Build Prototype → User Evaluates → Refine → (Cycle) → Final Product**

**Advantages of Prototyping Model:**

- Reduces risk of misunderstood requirements.
- User gets to see the system early and give feedback.
- Increases user satisfaction and involvement.
- Useful when requirements are unclear.

---

## Q22. Explain the Incremental Model.

The Incremental Model divides the software into increments (builds). Each increment delivers a working portion of the system and adds more functionality.

**Key Features:**

- Software is developed and delivered in pieces (increments).
- Core functionality is delivered in the first increment.

- Each subsequent increment adds more features.
- Customers can use partial system early.

**Advantages:**

- Early delivery of working software.
- Lower initial cost.
- Easier to test and debug smaller increments.

**Limitations:**

- Good architecture planning is essential upfront.
- Total cost may be higher than waterfall for some projects.

---

## Q23. Explain the Evolutionary Model.

The Evolutionary Model builds an initial version, exposes it to user feedback, and then refines it through multiple iterations. Unlike incremental model, the full system evolves with each iteration.

**Types:**

- Exploratory Development: Used when requirements are unclear; explore solutions iteratively.
- Throw-away Prototyping: Build prototype to understand requirements, then discard and rebuild properly.

**Characteristics:**

- Requirements evolve with the system.
- Each iteration produces an improved version.
- Suitable for large, complex, long-lived systems.

---

## Q24. Compare Incremental Model and Evolutionary Model.

| Incremental Model | Evolutionary Model |
|---|---|
| Requirements are known upfront | Requirements evolve with development |
| Each increment adds pre-planned features | Each iteration refines the entire system |
| Suitable when scope is clear | Suitable when scope is unclear or changes |
| Less risk of requirement misunderstanding | Higher adaptability to change |
| First increment delivers core functionality | First iteration is a working but incomplete system |

---

## Q25. Compare Prototyping Model and Waterfall Model.

| Prototyping Model | Waterfall Model |
|---|---|
| Used when requirements are unclear | Used when requirements are clear upfront |
| User involved throughout development | User involved mainly at requirements phase |
| Produces working prototype early | No working software until late stages |
| Flexible to requirement changes | Rigid — hard to accommodate changes |
| Risk of scope creep | Risk of discovering errors late |

---

## Q26. Differentiate between Verification and Validation with examples.

(See Q15 for the comparison table.)

**Examples:**

- Verification Example: Code review to check that code follows design specifications — static analysis without running the program.
- Validation Example: User Acceptance Testing (UAT) where actual users test the system to ensure it meets their real-world needs.

---

## Q27. Explain the concept of risk in software development models.

- Risk: A potential problem that could harm the project — may affect cost, schedule, or quality.

**Types of Risk:**

- Project Risk: Budget overrun, schedule delays, personnel issues.
- Technical Risk: Challenges in design, implementation, or technology.
- Business Risk: Market changes or product becoming obsolete.

**Risk in Models:**

- Waterfall: High risk — errors found late are expensive to fix.
- Prototyping: Reduces requirement risk but may increase scope risk.
- Spiral Model: Specifically designed for risk management — risk analysis at every iteration.

---

## Q28. What are the limitations of the Incremental Model?

- Requires a clear and complete definition of the whole system before it can be broken into increments.
- Poor architecture may result if increments are not properly planned.
- Total cost can be higher than developing the complete system at once.
- Integration problems may arise when combining increments.
- Not suitable for projects where requirements change frequently.

---

## Q29. When should Evolutionary Model be preferred?

- When requirements are not fully understood at the start.
- When the system needs to be delivered quickly and then improved.
- For research-based or innovative projects where the goal evolves.
- For large, complex, long-term projects that need continuous improvement.
- When there is significant user involvement and feedback loop is essential.

---

## Q30. Draw and explain the generic process framework.

The Generic Process Framework defines five core process activities applicable to all software projects:

- 1. Communication: Understand stakeholder objectives, requirements, and constraints.
- 2. Planning: Define tasks, milestones, timeline, risk, and resources.
- 3. Modeling: Create models (requirements, architectural, detailed design).
- 4. Construction: Generate code and perform testing.
- 5. Deployment: Deliver software to customer; get feedback.

■ *These five activities are present in every SDLC model — they may be sequential, iterative, or overlapping depending on the model used.*

# UNIT 3 — Agile and Modern Practices

## Q31. What is Agile Software Development?

Agile Software Development is an iterative and incremental approach to software development that emphasises flexibility, collaboration, customer satisfaction, and rapid delivery of working software. It emerged as a response to the limitations of heavyweight traditional models like Waterfall.

**Core Ideas:**

- Deliver working software in short cycles (sprints/iterations).
- Welcome changing requirements even late in development.
- Maintain close collaboration between developers and customers.
- Focus on individuals and interactions over processes and tools.

---

## Q32. List the principles of Agile methodology.

The Agile Manifesto (2001) defines 12 principles:

- Customer satisfaction through early and continuous delivery of valuable software.
- Welcome changing requirements, even late in development.
- Deliver working software frequently (weeks rather than months).
- Business people and developers must work together daily.
- Build projects around motivated individuals — give them the environment and support they need.
- Face-to-face conversation is the most efficient form of communication.
- Working software is the primary measure of progress.
- Agile processes promote sustainable development — maintain a constant pace.
- Continuous attention to technical excellence and good design enhances agility.
- Simplicity — maximising work NOT done — is essential.
- Best architectures and designs emerge from self-organising teams.
- The team regularly reflects on how to become more effective and adjusts accordingly.

---

## Q33. Explain the Agile development process.

- 1. Product Backlog Creation: All requirements are listed as user stories in a prioritised backlog.
- 2. Sprint Planning: Team selects items from backlog for the current sprint (1–4 weeks).
- 3. Sprint Execution: Team develops, tests, and integrates features daily.
- 4. Daily Scrum (Stand-up): 15-minute daily meeting — What did I do? What will I do? Any blockers?
- 5. Sprint Review: Team demonstrates working software to stakeholders.
- 6. Sprint Retrospective: Team reflects on process improvements.
- 7. Repeat: Next sprint begins with updated backlog.

---

## Q34. What is Extreme Programming (XP)? Explain TDD.

Extreme Programming (XP) is an Agile methodology focused on improving software quality and responsiveness to changing requirements through frequent releases in short development cycles.

**Core XP Practices:**

- Pair Programming: Two developers work on the same code simultaneously.
- Test-Driven Development (TDD): Write tests before writing code.
- Continuous Integration: Code is integrated and tested multiple times a day.
- Refactoring: Continuously improve code design without changing behaviour.

- Small Releases: Deliver small, frequent working releases.

**Test-Driven Development (TDD):**

- Step 1 — Write a failing test for the desired functionality.
- Step 2 — Write the minimum code to make the test pass.
- Step 3 — Refactor the code while keeping tests passing.
- Cycle: Red → Green → Refactor (repeated continuously).

■ *TDD ensures every piece of code has corresponding tests, leading to better design and fewer bugs.*

---

## Q35. Explain Scrum framework and lifecycle.

Scrum is an Agile framework for managing and completing complex projects. It uses fixed-length iterations called Sprints to deliver working software.

**Scrum Roles:**

- Product Owner: Maintains the product backlog; prioritises features.
- Scrum Master: Facilitates the process; removes impediments.
- Development Team: Cross-functional team that builds the product.

**Scrum Artefacts:**

- Product Backlog: Ordered list of all desired features.
- Sprint Backlog: Items selected for the current sprint.
- Increment: Working software produced at the end of each sprint.

**Scrum Events:**

- Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective.

**Scrum Lifecycle:**

- Product Backlog → Sprint Planning → Sprint (1–4 weeks) → Sprint Review → Sprint Retrospective → Repeat

---

## Q36. What are the advantages and disadvantages of Agile methodology?

**Advantages:**

- Working software delivered quickly and regularly.
- Highly adaptable to changing requirements.
- High customer involvement leads to better products.
- Early detection and fixing of defects.
- Motivated, self-organising teams.

**Disadvantages:**

- Difficult to predict cost and timeline upfront.
- Requires active and continuous customer involvement.
- Not suitable for very large, distributed teams without modifications.
- Documentation is often minimal.
- Scope creep can occur without strong product ownership.

---

## Q37. Explain the role of customer collaboration in Agile.

- Customers are active participants throughout development, not just at the start or end.
- They provide continuous feedback after each sprint/iteration.
- Customer priorities drive the product backlog.
- Daily stand-ups and sprint reviews ensure the customer's voice is heard.
- This collaboration reduces the risk of building the wrong product.
- The Agile Manifesto values 'Customer collaboration over contract negotiation'.

---

## Q38. Differentiate between Scrum and Extreme Programming (XP).

| Scrum | Extreme Programming (XP) |
|---|---|
| Project management framework | Software engineering methodology |
| Focuses on process and team structure | Focuses on engineering practices |
| Sprint length: 2–4 weeks (fixed) | Iteration length: 1–2 weeks (shorter) |
| No specific engineering practices mandated | Mandates TDD, pair programming, refactoring |
| Changes not allowed mid-sprint | More flexible — changes can be made if not started |
| Uses roles: PO, SM, Dev Team | No specific named roles |

---

## Q39. What is Sprint Review and Sprint Planning?

**Sprint Planning:**

- Held at the start of each sprint.
- Team selects items from the product backlog to work on.
- Team estimates effort and creates a sprint goal.
- Output: Sprint Backlog (tasks for the sprint).

**Sprint Review:**

- Held at the end of each sprint.
- Team demonstrates the completed increment to stakeholders.
- Product Owner accepts or rejects completed items.
- Feedback gathered to update the product backlog.

---

## Q40. Explain the concept of continuous integration in Agile.

Continuous Integration (CI) is an Agile practice where developers frequently merge their code changes into a shared repository — often multiple times a day. Each merge triggers an automated build and test process.

**Benefits:**

- Detects integration errors early and quickly.
- Reduces the time to find and fix bugs.
- Always maintains a working, tested codebase.
- Enables faster release cycles.
- Encourages smaller, manageable code commits.

■ *Popular CI tools: Jenkins, GitHub Actions, GitLab CI, CircleCI.*

# UNIT 4 — Cost Estimation (COCOMO)

## Q41. Explain the different modes of COCOMO (Organic, Semi-detached, Embedded).

COCOMO (Constructive Cost Model), proposed by Barry Boehm (1981), estimates software development effort and time based on Lines of Code (KLOC).

**The basic COCOMO formula:**

Effort (E) = $a \times (KLOC)^b$ | Duration (D) = $c \times (E)^d$

| Mode | Description | Example | a | b | c | d |
|---|---|---|---|---|---|---|
| Organic | Small teams, familiar environment, simple requirements | Payroll system | 2.4 | 1.05 | 2.5 | 0.38 |
| Semi-detached | Mixed experience teams, moderately complex | Database system | 3.0 | 1.12 | 2.5 | 0.35 |
| Embedded | Complex, hardware-integrated, strict constraints | ATM software, OS | 3.6 | 1.20 | 2.5 | 0.32 |

## Q42. What factors affect software cost estimation?

**Product Attributes:**

- Required software reliability.
- Size of application database.
- Complexity of the product.

**Hardware Attributes:**

- Performance constraints (CPU speed, memory).
- Target hardware platform.

**Personnel Attributes:**

- Analyst and programmer capability.
- Experience with programming language and development environment.

**Project Attributes:**

- Use of modern software tools and practices.
- Required development schedule.

## Q43. Differentiate between effort estimation and time estimation.

| Effort Estimation | Time Estimation |
|---|---|
| Measures total work required (person-months or person-hours) | Measures calendar time to complete the project |
| Depends on project size and complexity | Depends on effort and number of people available |
| Example: 120 person-months | Example: 10 months with 12 developers |
| Formula: E = a × (KLOC)<super>b</super> | Formula: D = c × (E)<super>d</super> |
| Does not consider team size | Considers parallel work and team size |

## Q44. Explain the limitations of COCOMO model.

- Based on Lines of Code (LOC) — difficult to estimate accurately early in a project.
- Does not account for modern development approaches like Agile, OOP, or reuse.

- Assumes a sequential waterfall-like process.
- Does not account for team communication overhead in large projects.
- Constants (a, b, c, d) were derived from 1970s projects and may be outdated.
- Does not consider non-coding activities like documentation and meetings.

---

## Q45. What is software cost estimation?

Software Cost Estimation is the process of predicting the effort, time, and resources (including money) required to develop a software system. It is a critical activity performed in the early stages of a project.

**Common Estimation Techniques:**

- Expert Judgment: Experienced professionals estimate based on past experience.
- Algorithmic Models: COCOMO, Function Point Analysis.
- Analogy-Based Estimation: Compare with similar past projects.
- Top-Down Estimation: Estimate overall cost, then break into components.
- Bottom-Up Estimation: Estimate each component individually, then sum up.

# UNIT 5 — Requirements Engineering

## Q46. What is SRS?

SRS (Software Requirements Specification) is a formal document that completely describes the behaviour of the software system to be developed. It serves as a contract between the customer and the development team.

**Contents of an SRS:**

- Introduction and purpose of the document.
- Overall description of the product.
- Specific functional requirements.
- Non-functional requirements (performance, security, usability).
- External interface requirements.
- System constraints and assumptions.

---

## Q47. Explain the purpose of SRS.

- Defines the scope of work and sets expectations between client and developer.
- Acts as the basis for project planning, design, and testing.
- Reduces development costs by clarifying requirements before coding begins.
- Serves as a reference for verification and validation activities.
- Used as a contractual basis between the client and the development organisation.
- Helps in identifying inconsistencies and gaps in requirements.

---

## Q48. List characteristics of a good SRS.

- Correct: Every stated requirement reflects an actual need of the system.
- Unambiguous: Each requirement has only one possible interpretation.
- Complete: All requirements are stated; no known requirement is missing.
- Consistent: No two requirements contradict each other.
- Ranked (Prioritised): Requirements are given an importance/stability ranking.
- Verifiable: Each requirement can be tested or verified.
- Modifiable: The SRS can be changed without affecting overall structure.
- Traceable: Each requirement can be traced to its source.

---

## Q49. Explain requirement gathering techniques.

- Interviews: One-on-one or group discussions with stakeholders to gather needs. Most common technique.
- Questionnaires/Surveys: Written questions distributed to a large group of stakeholders.
- Observation: Analyst watches users perform their tasks to identify actual workflow.
- Document Analysis: Reviewing existing system manuals, reports, and forms.
- Workshops / JAD Sessions: Joint Application Development sessions where stakeholders and developers collaborate.
- Brainstorming: Creative group technique to generate a wide range of ideas.
- Use Cases and User Stories: Describing system behaviour from the user's perspective.
- Prototyping: Building a rough model to help users articulate their requirements.

---

## Q50. What are Functional Requirements?

Functional Requirements describe what the system should DO — the functions, features, and behaviours the software must provide.

**Examples:**

- The system shall allow users to register with email and password.
- The system shall generate a monthly sales report in PDF format.
- The ATM shall dispense cash in multiples of ■100.
- The system shall send an OTP to the user's registered mobile number for login.

**Characteristics:**

- Directly related to the product's features.
- Captured as use cases or user stories.
- Usually testable with specific expected outputs.

---

## Q51. What are Non-Functional Requirements?

Non-Functional Requirements (NFRs) describe HOW the system performs its functions. They specify quality attributes of the system rather than specific behaviours.

**Types and Examples:**

- Performance: The system shall respond to user queries within 2 seconds.
- Reliability: The system shall be available 99.9% of the time (3-nines uptime).
- Security: All user passwords shall be stored using AES-256 encryption.
- Scalability: The system shall support up to 10,000 concurrent users.
- Usability: New users shall complete core tasks with less than 1 hour of training.
- Maintainability: The system shall allow module updates without full redeployment.
- Portability: The application shall run on Windows, macOS, and Linux.

---

## Q52. Differentiate Functional and Non-Functional Requirements.

| Functional Requirements | Non-Functional Requirements |
|---|---|
| Describe what the system should do | Describe how well the system should do it |
| Define specific behaviours and features | Define quality attributes and constraints |
| Captured as use cases, user stories | Captured as quality metrics and constraints |
| Testable by checking expected outputs | Tested through performance/security testing |
| Example: 'User can log in with email' | Example: 'Login response time < 1 second' |
| Directly visible to end users | Often invisible to users but critical to experience |
| Changes affect system functionality | Changes affect system architecture |

---

## Q53. Explain the importance of requirement analysis.

- Foundation of Development: All design and coding decisions are based on requirements — errors here propagate to all subsequent phases.
- Cost Reduction: Fixing requirement errors after coding is 100× more expensive than fixing them during analysis.
- Prevents Scope Creep: Clear requirements prevent uncontrolled addition of features.
- Customer Satisfaction: Ensures the delivered product matches customer expectations.
- Basis for Testing: Test cases are derived directly from requirements.
- Project Planning: Accurate estimation of cost, time, and resources depends on well-defined requirements.
- Risk Reduction: Identifies ambiguities and conflicts early, reducing project risk.

■ *Studies show that 40–60% of software defects originate in the requirements phase. Early requirement analysis is therefore the highest ROI activity in software engineering.*

**End of Notes — All 53 Questions Covered Across 5 Units**